

Arquitectura de Datos

Tema 3: Redis

Bases de datos clave-valor en memoria

Curso académico 2026/2027



PR5 Sesiones de usuario



Objetivo

En esta práctica aprenderás a implementar un sistema de **sesiones de usuario** utilizando **Redis** como almacenamiento clave-valor. Analizarás el uso de **TTL** (Time To Live), diseñarás estrategias de **invalidación** y concurrencia, y valorarás la **escalabilidad horizontal** en entornos de producción.

Al finalizar esta práctica serás capaz de:

- Diseñar el esquema de claves Redis para almacenar sesiones de usuario.
- Implementar las operaciones de login y logout con expiración automática.
- Analizar el comportamiento del TTL y su impacto en la experiencia de usuario.
- Detectar y gestionar sesiones duplicadas en múltiples dispositivos.
- Identificar y mitigar el problema del thundering herd.
- Implementar invalidación global de sesiones por eventos de seguridad.
- Proponer una arquitectura distribuida para escalar a miles de usuarios concurrentes.

Enunciado

Escenario:

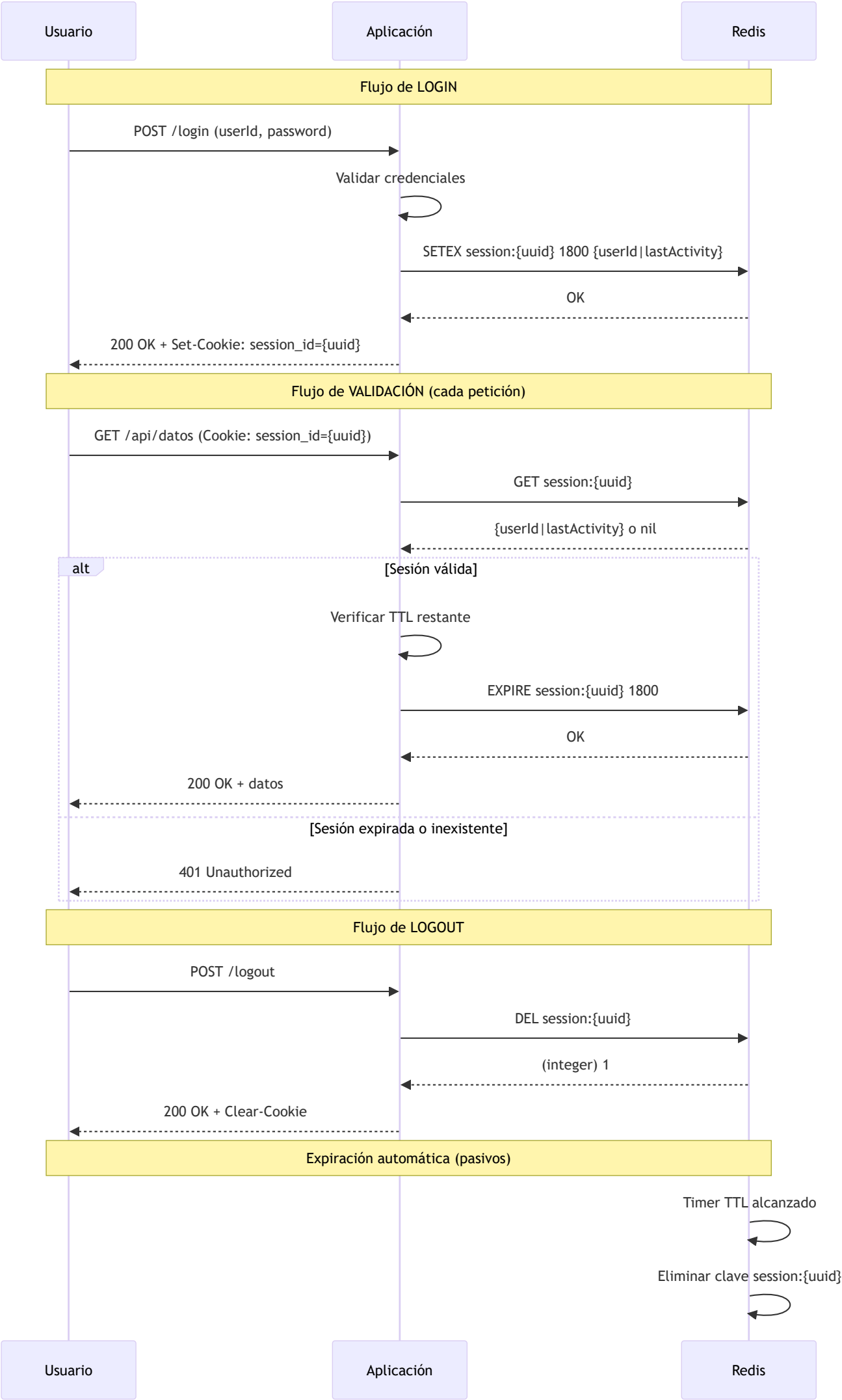
Eres el arquitecto de una plataforma web SaaS con **10.000 usuarios concurrentes**. Necesitas implementar un sistema de sesiones de usuario con Redis que soporte:

- **Login / Logout:** creación y destrucción controlada de sesiones.
- **Expiración automática:** las sesiones inactivas deben eliminarse tras un período configurable.
- **Invalidación por seguridad:** cambio de contraseña o detección de intrusión debe cerrar todas las sesiones del usuario.
- **Escalabilidad horizontal:** el sistema debe funcionar cuando la carga supere la capacidad de un único nodo Redis.

Diseña e implementa el sistema completo, justificando cada decisión técnica.

Diagrama del escenario

El siguiente diagrama de secuencia muestra el flujo típico de una sesión de usuario: creación, validación y expiración.



Explicación: Cada petición del usuario renueva el TTL de la sesión (sliding expiration). Si el usuario no realiza ninguna petición en 30 minutos, Redis elimina la clave automáticamente y la sesión caduca.

Requisitos previos

Componente	Versión mínima	Comprobación
Docker Desktop	4.x	<code>docker --version</code>
JetBrains DataGrip	2024.x	Abrir la aplicación
.NET SDK	10.0	<code>dotnet --version</code>
Python	3.12+	<code>python --version</code>
Práctica PR3 completada	—	Cache-Aside y sesiones básicas
Contenidos Sesión 2	T3.2.4	Convenciones de nombrado
Contenidos Sesión 3	T3.3.5	Patrones de caché

Nota: Esta práctica asume que ya has completado la Práctica PR3 (Cache-Aside y sesiones). Si no lo has hecho, revisa los contenidos de la Sesión 3 sobre patrones de caché antes de comenzar.

Tarea 1: Diseñar el esquema de sesiones

Enunciado:

Diseña el patrón de claves Redis para almacenar sesiones de usuario. Tu esquema debe incluir:

- Identificador único de sesión (session ID).
- Datos del usuario asociados a la sesión.
- Timestamp de última actividad.
- TTL configurado para expiración automática.

Justifica cada decisión de diseño. Por qué ese prefijo, por qué ese separador, por qué ese tipo de dato, etc.

Pregunta de investigación:

¿Qué convención de nombrado usarías para las claves? ¿Almacenarías los datos de la sesión en una sola clave serializada (JSON) o en múltiples claves? ¿Qué ventajas e inconvenientes tiene cada enfoque?

Pista: Consulta la **Sesión 2, T3.2.4** sobre convenciones de nombrado en bases de datos clave-valor, y la **Sesión 3, T3.3.5** sobre patrones de caché para decidir entre serialización completa vs. campos individuales.

Tarea 2: Implementar login / logout

Enunciado:

Implementa las funciones `login(userId, password)` y `logout(sessionId)` en tu lenguaje preferido (.NET o Python).

- El **login** debe crear una sesión con TTL de 30 minutos.
- El **logout** debe eliminar la sesión de Redis.
- Debes generar un identificador de sesión único y seguro (UUID v4 o similar).

¿Qué estructura de datos usarías para representar la sesión dentro de Redis?

Pregunta de investigación:

¿Usarías SETEX para crear la sesión con expiración, o preferirías SET seguido de EXPIRE? ¿Qué diferencia hay en términos de atomicidad? ¿Cómo serializarías los datos de usuario: JSON, Hash, o campos separados?

Pista: Consulta la **Práctica PR3** sobre el patrón Cache-Aside y cómo se aplican las operaciones de sesión, y la **Práctica PR2** sobre estructuras de datos de Redis (Strings, Hashes, Sets) para elegir la representación más adecuada.

Tarea 3: Analizar expiración

Enunciado:

Un usuario tiene una sesión activa con TTL de 30 minutos. A los **29 minutos**, realiza una petición al servidor.

- ¿Qué debería pasar con el TTL de la sesión?
- ¿Cómo implementarías la renovación del TTL?
- ¿Qué diferencia hay entre **sliding expiration** y **fixed expiration**?
- Razona las ventajas e inconvenientes de cada estrategia.

Pregunta de investigación:

En **sliding expiration**, cada petición renueva el TTL. En **fixed expiration**, la sesión caduca a una hora fija independientemente de la actividad. ¿Cuándo usarías cada una? ¿Qué impacto tiene en la seguridad y en la experiencia de usuario?

Pista: Consulta la **Sesión 3, T3.3.3** sobre invalidación de caché, donde se analizan las estrategias de expiración y sus implicaciones en sistemas de alta disponibilidad.

Tarea 4: Sesiones duplicadas

Enunciado:

Un usuario inicia sesión desde el **ordenador** y 5 minutos después desde el **móvil**.

- ¿Deberías permitir ambas sesiones simultáneamente?
- ¿Cómo detectarías que un usuario tiene múltiples sesiones activas?
- ¿Cómo manejarías el caso de que el usuario cambie de contraseña desde el móvil? ¿Se cerrarían las sesiones del ordenador?
- ¿Qué política de sesiones múltiples implementarías en una aplicación bancaria vs. una red social?

Pregunta de investigación:

Para rastrear todas las sesiones activas de un usuario, ¿qué estructura de datos de Redis usarías? ¿Un Set por usuario con todos los session IDs? ¿Cómo gestionarías la limpieza de sesiones caducadas de esa estructura?

Pista: Consulta la **Sesión 2** sobre Sets y Sorted Sets de Redis. Los Sets permiten almacenar elementos únicos y verificar pertenencia en $O(1)$, lo cual es ideal para rastrear sesiones activas por usuario.

Tarea 5: Thundering herd

Enunciado:

10.000 usuarios tienen sesiones que expiran a las **12:00:00** exactamente. Todos intentan re-login simultáneamente al detectar que su sesión ha caducado.

- ¿Qué problema de rendimiento aparece en este escenario?
- ¿Cómo se llama este fenómeno y por qué es perjudicial?
- ¿Qué estrategias de mitigación existen?
- ¿Cómo las implementarías en Redis?

Pregunta de investigación:

El **thundering herd** (manada rugiente) ocurre cuando miles de clientes esperan un evento que ocurre simultáneamente. Piensa en: ¿cómo distribuirías las expiraciones en el tiempo para evitar picos de carga? ¿Qué papel juega el **jitter** (aleatorización) en la solución?

Pista: Consulta la **Sesión 3, T3.3.3** sobre thundering herd. La solución típica consiste en añadir un componente aleatorio (jitter) al TTL base para distribuir las expiraciones en el tiempo y evitar que todos los usuarios expiren en el mismo instante.

Tarea 6: Invalidación por seguridad

Enunciado:

Un usuario cambia su contraseña desde otro dispositivo. Necesitas **invalidar TODAS las sesiones activas** de ese usuario en todos los dispositivos.

- ¿Cómo almacenarías la relación entre usuario y sus sesiones activas?
- ¿Qué comandos de Redis ejecutarías para invalidar todas las sesiones de un usuario?
- ¿Cómo manejarías la sesión que acaba de cambiar la contraseña (no debería invalidarse a sí misma)?
- ¿Qué pasa si un usuario tiene 50 sesiones activas? ¿Es eficiente eliminarlas una por una?

Pregunta de investigación:

Para invalidar todas las sesiones de un usuario, necesitas una estructura que asocie cada `userId` con el conjunto de sus `sessionIds`. Después, ejecutarías `DEL` sobre cada `sessionId`. ¿Qué alternativas existen para hacer esto de forma atómica? ¿Cómo usarías un Lua script de Redis?

Pista: Consulta la **Sesión 3** sobre claves compuestas y la **Práctica PR3** sobre invalidación de caché. Un patrón común es mantener un Set por usuario con todos sus session IDs, y usar un Lua script para ejecutar la invalidación de forma atómica.

Tarea 7: Escalabilidad

Enunciado:

Tu plataforma crece a **100.000 usuarios concurrentes**. Redis en un solo nodo no da abasto: la memoria RAM se agota y la latencia aumenta.

- ¿Qué arquitectura distribuida propondrías para las sesiones?
- ¿Cómo partitionarías las sesiones entre múltiples nodos Redis?
- ¿Qué estrategia de **sharding** usarías? ¿Por qué?
- ¿Cómo manejarías la alta disponibilidad si un nodo Redis falla?
- ¿Qué métricas monitorearías para detectar problemas de escalabilidad?

Pregunta de investigación:

Redis Cluster ofrece sharding automático basado en **hash slots** (16.384 slots distribuidos entre los nodos). ¿Cómo afecta el sharding a las operaciones que involucran múltiples claves (como invalidar todas las sesiones de un usuario)? ¿Qué es el **cross-slot** y por qué es problemático?

Pista: Consulta la **Sesión 4, T3.4.2** sobre Redis Cluster. Analiza cómo Redis distribuye las claves mediante hash slots, cómo se gestionan las réplicas para alta disponibilidad, y cuándo es necesario usar **hash tags** para forzar que claves relacionadas estén en el mismo nodo.

Preguntas de reflexión

Antes de finalizar la práctica, reflexiona sobre las siguientes preguntas. No necesitas responderlas por escrito, pero debes ser capaz de justificar tus decisiones arquitectónicas en una entrevista técnica o en una revisión de código.

1. ¿Por qué usar TTL en sesiones y no dejarlas indefinidas?

Considera las implicaciones de seguridad, el uso de memoria y la experiencia de usuario. ¿Qué pasaría si un usuario olvida cerrar sesión en un ordenador público?

2. ¿Qué diferencia hay entre expirar por TTL y invalidar por evento?

El TTL elimina la sesión tras un período de inactividad. La invalidación por evento (cambio de contraseña, cierre remoto) elimina la sesión de forma inmediata. ¿Cuándo necesitas ambas estrategias?

3. ¿Cómo afecta la pérdida de Redis a los usuarios con sesión activa?

Si Redis se reinicia sin persistencia, todas las sesiones se pierden. Los usuarios deberían volver a iniciar sesión. ¿Qué mecanismos de persistencia (AOF, RDB) usarías? ¿Qué nivel de pérdida de datos es aceptable?

4. ¿Qué estrategia de caché usarías para sesiones?

¿Cache-Aside (la aplicación gestiona la caché), Write-Through (se escribe en caché y base de datos simultáneamente) o Write-Behind (se escribe en caché y se difiere la escritura en base de datos)? Justifica tu respuesta.

5. ¿Cómo equilibrar seguridad (sesiones cortas) y usabilidad (sesiones largas)?

Una sesión de 5 minutos es más segura pero frustrante. Una sesión de 24 horas es cómoda pero arriesgada. ¿Qué TTL elegirías para diferentes tipos de aplicación (banca, redes sociales, correo electrónico)?

6. ¿Qué pasaría si usas SET en lugar de SETEX para sesiones?

Sin TTL, las sesiones nunca caducan automáticamente. ¿Cómo las eliminarías? ¿Qué problema de memoria surgiría con el tiempo? ¿Necesitarías un job de limpieza periódico?

7. ¿Cómo monitorizarías el número de sesiones activas?

Redis no proporciona un contador automático de claves con un patrón específico. ¿Qué estrategia usarías para contar sesiones activas en tiempo real? ¿Usarías un contador atómico con INCR/DECR?

8. ¿Qué métricas de Redis indicarían problemas con las sesiones?

Piensa en: uso de memoria (INFO memory), número de claves (INFO keyspace), latencia de comandos (--latency), tasa de evicciones (evicted_keys). ¿Qué umbrales configurarías para cada alerta?



Arquitectura de Datos

4.º Curso, Grado en Ingeniería Informática

Curso académico 2026/2027

Human Language and Accessibility Technologies Group (HULAT).

Departamento de Informática

Universidad Carlos III de Madrid